

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP
KHOA ĐIỆN TỬ

Bộ môn: Công nghệ Thông tin



BÀI TIỂU LUẬN
ĐẢM BẢO CHẤT LƯỢNG VÀ
KIỂM THỬ PHẦN MỀM
ĐỀ TÀI: LLM TRONG PHÂN TÍCH YÊU CẦU VÀ
SINH CA KIỂM THỬ

GIÁO VIÊN HƯỚNG DẪN : TS. NGUYỄN TUẤN LINH
HỌ VÀ TÊN SINH VIÊN : ĐẬU VĂN KHÁNH
LỚP : K58KTP
MSSV : K225480106099

THÁI NGUYÊN - 2026

TRƯỜNG ĐHKTCN

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM

KHOA ĐIỆN TỬ

Độc lập - Tự do - Hạnh phúc

BÀI TIỂU LUẬN

MÔN HỌC: ĐẢM BẢO CHẤT LƯỢNG VÀ KIỂM THỬ PHẦN MỀM

BỘ MÔN : CÔNG NGHỆ THÔNG TIN

Sinh viên: Đặng Văn Khánh

MSSV: K225480106099

Lớp: K58KTP

Ngành: Kỹ thuật máy tính

Giáo viên hướng dẫn: TS. Nguyễn Tuấn Linh

Ngày giao đề: 03/06/2026

Ngày hoàn thành: 24/06/2026

Tên đề tài : LLM trong phân tích yêu cầu và sinh ca kiểm thử.

Yêu cầu:

- Nghiên cứu cơ sở lý thuyết về LLM, trọng tâm là kiến trúc Transformer và cơ chế Self-Attention.
- Phân tích khả năng của LLM trong việc đọc hiểu và trích xuất quy tắc nghiệp vụ từ tài liệu văn bản.
- Xây dựng quy trình sử dụng kỹ thuật Prompt Engineering để tự động sinh các hiện vật kiểm thử (Ma trận truy vết, Bảng quyết định, kịch bản PyTest).
- Vận dụng giải pháp vào bài toán thực tế (hệ thống đăng nhập Flask API) để kiểm chứng.
- Đánh giá định lượng hiệu quả về độ bao phủ yêu cầu, thời gian thực hiện và độ chính xác.
- Phân tích rủi ro (như hiện tượng hallucination) và đề xuất hướng tối ưu hóa kết quả.

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày... tháng... năm 2026

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

MỤC LỤC

LỜI CẢM ƠN	6
DANH MỤC CÁC BẢNG VÀ HÌNH ẢNH	7
LỜI NÓI ĐẦU	8
CHƯƠNG 1: MỞ ĐẦU	9
1.1. Lý do chọn đề tài	9
1.2. Mục tiêu nghiên cứu	9
1.3. Phạm vi và đối tượng nghiên cứu	10
1.3.1. Phạm vi nghiên cứu	10
1.3.2. Đối tượng nghiên cứu	12
1.4. Phương pháp nghiên cứu	12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN	14
2.1. Định nghĩa và các khái niệm cốt lõi	14
2.1.1. Mô hình ngôn ngữ lớn (Large Language Models – LLM)	14
2.1.2. Kiến trúc Transformer và nguyên lý hoạt động của LLM	15
2.1.3. Phân tích yêu cầu phần mềm (Requirements Analysis)	16
2.1.4. Kiểm thử phần mềm và ca kiểm thử	16
2.1.5. Các loại hiện vật kiểm thử (Test Artifacts)	18
2.2. Phân tích chuyên sâu	18
2.2.1. Cơ chế hoạt động của LLM trong phân tích yêu cầu	18
2.2.2. Cơ chế sinh ca kiểm thử bằng LLM	19
2.2.3. Prompt Engineering trong phân tích yêu cầu và sinh ca kiểm thử	19
2.2.4. Retrieval-Augmented Generation (RAG) trong hỗ trợ kiểm thử phần mềm	21
2.2.5. Ưu điểm của việc ứng dụng LLM	21
2.2.6. Hạn chế và rủi ro của LLM	22
2.2.7. So sánh phương pháp truyền thống và phương pháp sử dụng LLM	22
2.2.8. Quy trình ứng dụng LLM trong phân tích yêu cầu và sinh ca kiểm thử	23
2.3. Sự liên hệ với chất lượng phần mềm	24

CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ	25
3.1. Mô tả bài toán thực tế	25
3.1.1. Bài toán thực tế	25
3.1.2. Đặc tả yêu cầu hệ thống	25
3.1.3. Kiến trúc hệ thống thực nghiệm	26
3.1.4. Cấu trúc thư mục dự án	27
3.2. Áp dụng giải pháp	28
3.2.1. Quy trình áp dụng LLM	28
3.2.2. Thiết kế Prompt cho LLM	28
3.2.3. Phân tích yêu cầu bằng LLM	29
3.2.4. Xây dựng ma trận truy vết yêu cầu (Traceability Matrix)	30
3.2.5. Thiết kế bảng quyết định (Decision Table)	30
3.2.6. Thiết kế kiểm thử giá trị biên	31
3.2.7. Sinh Test Case tự động	31
3.2.8. Xây dựng kịch bản kiểm thử tự động	31
3.2.9. Thực thi kiểm thử và vận hành hệ thống	32
3.3. Kết quả và đánh giá	34
3.3.1. Kết quả đạt được	34
3.3.2. Đánh giá độ bao phủ yêu cầu	34
3.3.3. So sánh hiệu quả trước và sau khi sử dụng LLM	35
3.3.4. Đánh giá rủi ro và hạn chế	35
3.3.5. Đánh giá chi phí và khả năng ứng dụng	36
CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ	37
4.1. Tổng kết nội dung nghiên cứu	37
4.2. Hạn chế của giải pháp	37
4.3. Định hướng mở rộng	38
TÀI LIỆU THAM KHẢO	39

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn sâu sắc đến thầy **Nguyễn Tuấn Linh** giảng viên Trường Đại học Kỹ thuật Công nghiệp Thái Nguyên, người đã tận tình hướng dẫn, chỉ bảo và đồng hành cùng em trong suốt quá trình thực hiện và hoàn thiện báo cáo bài tiểu luận này. Nhờ sự giúp đỡ quý báu của thầy, em đã có thể vượt qua những khó khăn, từng bước hoàn thành tốt nội dung đề tài được giao.

Mặc dù đã cố gắng hết sức, nhưng trong quá trình thực hiện báo cáo, chắc chắn sẽ không tránh khỏi những thiếu sót nhất định. Em rất mong nhận được sự thông cảm và những góp ý quý báu từ các thầy, cô để em có thể rút kinh nghiệm, hoàn thiện bản thân hơn trong những chặng đường học tập và công tác sau này.

Em xin chân thành cảm ơn!

DANH MỤC CÁC BẢNG VÀ HÌNH ẢNH

Bảng 2.1. So sánh giữa phương pháp truyền thống và phương pháp sử dụng LLM	22
Bảng 3.1: Xây dựng ma trận truy vết yêu cầu	30
Bảng 3.2: Bảng quyết định	30
Bảng 3.3: Sinh Test Case tự động	31
Bảng 3.4: So sánh hiệu quả trước và sau khi sử dụng LLM	35
Hình 2.1: Kiến trúc tổng quát của mô hình ngôn ngữ lớn (LLM)	14
Hình 2.2: Kiến trúc Transformer	15
Hình 2.3: Cấu trúc cơ bản của một ca kiểm thử (Test Case)	17
Hình 2.4: Vai trò của Prompt Engineering trong ứng dụng LLM	20
Hình 2.5: Kiến trúc Retrieval-Augmented Generation (RAG) trong hệ thống LLM	21
Hình 2.6. Quy trình ứng dụng LLM trong phân tích yêu cầu và sinh ca kiểm thử	23
Hình 3.1: Tài liệu yêu cầu đầu vào	26
Hình 3.2. Kiến trúc hệ thống thực nghiệm	27
Hình 3.3. Cấu trúc thư mục dự án	27
Hình 3.4: Quy trình áp dụng LLM	28
Hình 3.5. Minh họa Prompt Engineering	29
Hình 3.6. Cấu trúc bộ kiểm thử tự động	32
Hình 3.7. Kết quả khởi động hệ thống Flask API	32
Hình 3.8. Kết quả truy cập API	33
Hình 3.9. Kết quả thực thi PyTest	33

LỜI NÓI ĐẦU

Trong bối cảnh chuyển đổi số và sự phát triển mạnh mẽ của công nghệ thông tin, chất lượng phần mềm ngày càng trở thành yếu tố quan trọng quyết định hiệu quả hoạt động của các hệ thống thông tin. Để đảm bảo chất lượng sản phẩm, hoạt động kiểm thử phần mềm đóng vai trò không thể thiếu trong toàn bộ vòng đời phát triển phần mềm. Tuy nhiên, việc phân tích yêu cầu và xây dựng các ca kiểm thử thường tốn nhiều thời gian, công sức và phụ thuộc lớn vào kinh nghiệm của đội ngũ kiểm thử.

Sự ra đời của các mô hình ngôn ngữ lớn (Large Language Models - LLM) như GPT, Gemini và Claude đã mở ra nhiều cơ hội mới trong lĩnh vực kỹ nghệ phần mềm. Với khả năng hiểu và xử lý ngôn ngữ tự nhiên, các mô hình này có thể hỗ trợ phân tích tài liệu yêu cầu, trích xuất thông tin nghiệp vụ và tự động sinh các ca kiểm thử, góp phần nâng cao hiệu quả và năng suất của hoạt động kiểm thử phần mềm.

Xuất phát từ thực tiễn đó, đề tài “LLM trong phân tích yêu cầu và sinh ca kiểm thử” được thực hiện nhằm nghiên cứu khả năng ứng dụng các mô hình ngôn ngữ lớn trong quá trình phân tích yêu cầu phần mềm và thiết kế ca kiểm thử tự động. Đồng thời, đề tài cũng đánh giá những lợi ích, hạn chế và các thách thức khi áp dụng LLM trong kiểm thử phần mềm, từ đó đề xuất các giải pháp nhằm nâng cao độ chính xác và độ tin cậy của kết quả đầu ra.

Thông qua việc nghiên cứu và thực nghiệm, đề tài góp phần làm rõ tiềm năng của trí tuệ nhân tạo trong việc hỗ trợ đảm bảo chất lượng phần mềm, đồng thời cung cấp cơ sở tham khảo cho việc ứng dụng LLM trong các dự án phát triển phần mềm hiện nay.

CHƯƠNG 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong những năm gần đây, trí tuệ nhân tạo (Artificial Intelligence - AI) đã trở thành một trong những công nghệ có tốc độ phát triển nhanh nhất trong lĩnh vực công nghệ thông tin. Đặc biệt, sự xuất hiện của các mô hình ngôn ngữ lớn (Large Language Model - LLM) như GPT, Gemini, Claude hay Llama đã mở ra nhiều cơ hội mới trong quá trình phát triển phần mềm.

Trong quy trình phát triển phần mềm hiện đại, việc phân tích yêu cầu và thiết kế ca kiểm thử đóng vai trò rất quan trọng đối với chất lượng sản phẩm. Tuy nhiên, đây cũng là những hoạt động tiêu tốn nhiều thời gian và nguồn lực. Các nhóm kiểm thử thường phải đọc tài liệu đặc tả, xác định điều kiện nghiệp vụ, xây dựng test case và đánh giá rủi ro một cách thủ công. Quá trình này dễ xảy ra sai sót do yêu cầu không đầy đủ, mơ hồ hoặc thay đổi liên tục.

Sự phát triển của LLM mang đến khả năng xử lý ngôn ngữ tự nhiên ở mức độ cao, cho phép tự động phân tích tài liệu yêu cầu, nhận diện điều kiện kiểm thử, sinh test case và hỗ trợ đánh giá rủi ro. Điều này giúp giảm đáng kể công sức của tester, đồng thời nâng cao tốc độ và hiệu quả kiểm thử.

Bên cạnh những lợi ích rõ rệt, việc ứng dụng LLM trong kiểm thử phần mềm cũng đặt ra nhiều thách thức như độ chính xác của đầu ra, hiện tượng hallucination, thiếu khả năng giải thích kết quả và các vấn đề liên quan đến bảo mật dữ liệu.

Xuất phát từ những lý do trên, đề tài “LLM trong phân tích yêu cầu và sinh ca kiểm thử” được lựa chọn nhằm nghiên cứu khả năng ứng dụng LLM trong hoạt động kiểm thử phần mềm, từ đó đề xuất quy trình khai thác hiệu quả công nghệ này trong thực tiễn.

1.2. Mục tiêu nghiên cứu

❖ Mục tiêu tổng quát:

- Nghiên cứu khả năng ứng dụng các mô hình ngôn ngữ lớn (Large Language Models - LLM) trong quá trình phân tích yêu cầu phần mềm và sinh ca kiểm thử tự động, từ đó đánh giá hiệu quả, lợi ích, hạn chế và đề xuất quy trình áp dụng phù hợp nhằm nâng cao chất lượng kiểm thử phần mềm.

❖ Mục tiêu cụ thể:

Để đạt được mục tiêu tổng quát trên, đề tài tập trung thực hiện các mục tiêu cụ thể sau:

- Nghiên cứu tổng quan về mô hình ngôn ngữ lớn (LLM), bao gồm khái niệm, nguyên lý hoạt động và các mô hình phổ biến hiện nay.
- Tìm hiểu vai trò của LLM trong vòng đời phát triển phần mềm, đặc biệt là trong hoạt động kiểm thử phần mềm.
- Phân tích khả năng sử dụng LLM để hỗ trợ phân tích yêu cầu phần mềm từ các tài liệu đặc tả, User Story và Use Case.
- Nghiên cứu cách thức LLM trích xuất các yêu cầu chức năng, quy tắc nghiệp vụ và điều kiện kiểm thử từ tài liệu yêu cầu.
- Phân tích khả năng sinh ca kiểm thử tự động bằng LLM đối với các loại kiểm thử chức năng, kiểm thử biên và kiểm thử ngoại lệ.
- Xây dựng quy trình ứng dụng LLM trong hoạt động phân tích yêu cầu và thiết kế ca kiểm thử.
- Đánh giá những lợi ích của việc sử dụng LLM như tiết kiệm thời gian, nâng cao năng suất và tăng độ bao phủ kiểm thử.
- Phân tích các hạn chế và rủi ro khi ứng dụng LLM trong kiểm thử phần mềm như hiện tượng hallucination, thiếu hiểu biết nghiệp vụ và vấn đề bảo mật dữ liệu.
- Đề xuất các biện pháp kiểm chứng đầu ra nhằm nâng cao độ chính xác và độ tin cậy của các kết quả được sinh ra bởi LLM.

1.3. Phạm vi và đối tượng nghiên cứu

1.3.1. Phạm vi nghiên cứu

Để đảm bảo tính tập trung và phù hợp với mục tiêu của đề tài, phạm vi nghiên cứu được xác định như sau:

❖ Về nội dung nghiên cứu:

- Nghiên cứu các khái niệm, đặc điểm và nguyên lý hoạt động của mô hình ngôn ngữ lớn (Large Language Models - LLM).
- Nghiên cứu quy trình phân tích yêu cầu phần mềm trong các dự án phát triển phần mềm hiện đại.

- Nghiên cứu các kỹ thuật thiết kế và xây dựng ca kiểm thử dựa trên yêu cầu phần mềm.
 - Tìm hiểu khả năng ứng dụng của LLM trong việc hỗ trợ phân tích yêu cầu và sinh ca kiểm thử tự động.
 - Nghiên cứu các phương pháp đánh giá và kiểm chứng chất lượng đầu ra của LLM trong hoạt động kiểm thử phần mềm.
- ❖ Về công nghệ nghiên cứu:
- Khảo sát các mô hình ngôn ngữ lớn phổ biến hiện nay như GPT, Gemini và Claude.
 - Tìm hiểu khả năng ứng dụng các công cụ AI tạo sinh trong việc hỗ trợ hoạt động kiểm thử phần mềm.
 - Đánh giá hiệu quả của các mô hình LLM trong việc xử lý ngôn ngữ tự nhiên và sinh nội dung phục vụ kiểm thử.
- ❖ Về đối tượng áp dụng:
- Các tài liệu yêu cầu phần mềm được mô tả dưới dạng User Story, Use Case và tài liệu đặc tả yêu cầu phần mềm (SRS).
 - Các hệ thống phần mềm quản lý hoặc ứng dụng web có chức năng nghiệp vụ phổ biến.
 - Các hoạt động phân tích yêu cầu và thiết kế ca kiểm thử trong quy trình phát triển phần mềm.
- ❖ Về phạm vi kiểm thử:
- Tập trung nghiên cứu kiểm thử chức năng (Functional Testing).
 - Xem xét các loại ca kiểm thử như:
 - Kiểm thử hợp lệ (Positive Test Cases).
 - Kiểm thử không hợp lệ (Negative Test Cases).
 - Kiểm thử giá trị biên (Boundary Value Testing).
 - Kiểm thử ngoại lệ (Exception Testing).
- ❖ Nội dung không thuộc phạm vi nghiên cứu:
- Không nghiên cứu quá trình huấn luyện hoặc tinh chỉnh mô hình AI.
 - Không đi sâu vào kiến trúc Transformer và các thuật toán học sâu.
 - Không nghiên cứu phát triển mô hình ngôn ngữ mới.

- Không nghiên cứu các loại kiểm thử chuyên sâu như kiểm thử hiệu năng, kiểm thử tải, kiểm thử bảo mật nâng cao hoặc các kỹ thuật học máy khác.

1.3.2. Đối tượng nghiên cứu

- ❖ Các mô hình ngôn ngữ lớn (Large Language Models - LLM) như GPT, Gemini và Claude.
- ❖ Các tài liệu yêu cầu phần mềm dưới dạng User Story, Use Case và Software Requirement Specification (SRS).
- ❖ Quy trình phân tích yêu cầu phần mềm trong giai đoạn đầu của vòng đời phát triển phần mềm.
- ❖ Các kỹ thuật thiết kế và xây dựng ca kiểm thử dựa trên yêu cầu phần mềm.
- ❖ Khả năng ứng dụng LLM trong việc hỗ trợ phân tích yêu cầu, trích xuất quy tắc nghiệp vụ và sinh ca kiểm thử tự động.
- ❖ Chất lượng của các ca kiểm thử được tạo ra bởi LLM thông qua các tiêu chí như độ chính xác, độ đầy đủ, khả năng bao phủ yêu cầu và khả năng truy vết.
- ❖ Các lợi ích, hạn chế và rủi ro khi áp dụng LLM vào hoạt động kiểm thử phần mềm trong thực tế.

1.4. Phương pháp nghiên cứu

Để đạt được các mục tiêu nghiên cứu đã đề ra, đề tài sử dụng kết hợp nhiều phương pháp nghiên cứu nhằm đảm bảo tính khoa học, khách quan và độ tin cậy của kết quả:

- Phương pháp nghiên cứu tài liệu: Thu thập, phân tích các tài liệu, giáo trình, bài báo khoa học liên quan đến LLM, kỹ nghệ phần mềm, phân tích yêu cầu và kiểm thử phần mềm nhằm xây dựng cơ sở lý thuyết.
- Phương pháp phân tích và tổng hợp: Nghiên cứu nguyên lý hoạt động, khả năng và hạn chế của LLM; tổng hợp các phương pháp ứng dụng AI trong kiểm thử phần mềm.
- Phương pháp nghiên cứu tình huống (Case Study): Xây dựng hệ thống thực nghiệm, sử dụng LLM để phân tích yêu cầu, trích xuất quy tắc nghiệp vụ và sinh Test Case.

- Phương pháp so sánh và đánh giá: Đánh giá hiệu quả của Test Case sinh bởi LLM dựa trên độ chính xác, khả năng bao phủ yêu cầu, khả năng truy vết và thời gian thực hiện.
- Phương pháp thực nghiệm: Thiết kế Prompt, tiến hành thử nghiệm với các yêu cầu phần mềm cụ thể và phân tích kết quả để đánh giá khả năng ứng dụng của LLM trong kiểm thử.
- Phương pháp đánh giá định tính: Phân tích lợi ích, hạn chế và các rủi ro của việc sử dụng LLM như sai lệch ngữ cảnh, hiện tượng sinh thông tin không chính xác và vấn đề bảo mật dữ liệu.

Tóm tắt chương 1: Chương 1 đã trình bày tổng quan về đề tài ứng dụng LLM trong phân tích yêu cầu và sinh ca kiểm thử, bao gồm lý do chọn đề tài, mục tiêu, phạm vi, đối tượng và phương pháp nghiên cứu. Đề tài tập trung nghiên cứu khả năng ứng dụng LLM trong việc hỗ trợ phân tích yêu cầu, trích xuất quy tắc nghiệp vụ và tự động sinh Test Case nhằm nâng cao hiệu quả kiểm thử phần mềm. Các phương pháp nghiên cứu như nghiên cứu tài liệu, thực nghiệm, phân tích và đánh giá được sử dụng làm cơ sở cho quá trình triển khai và kiểm chứng giải pháp trong các chương tiếp theo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN

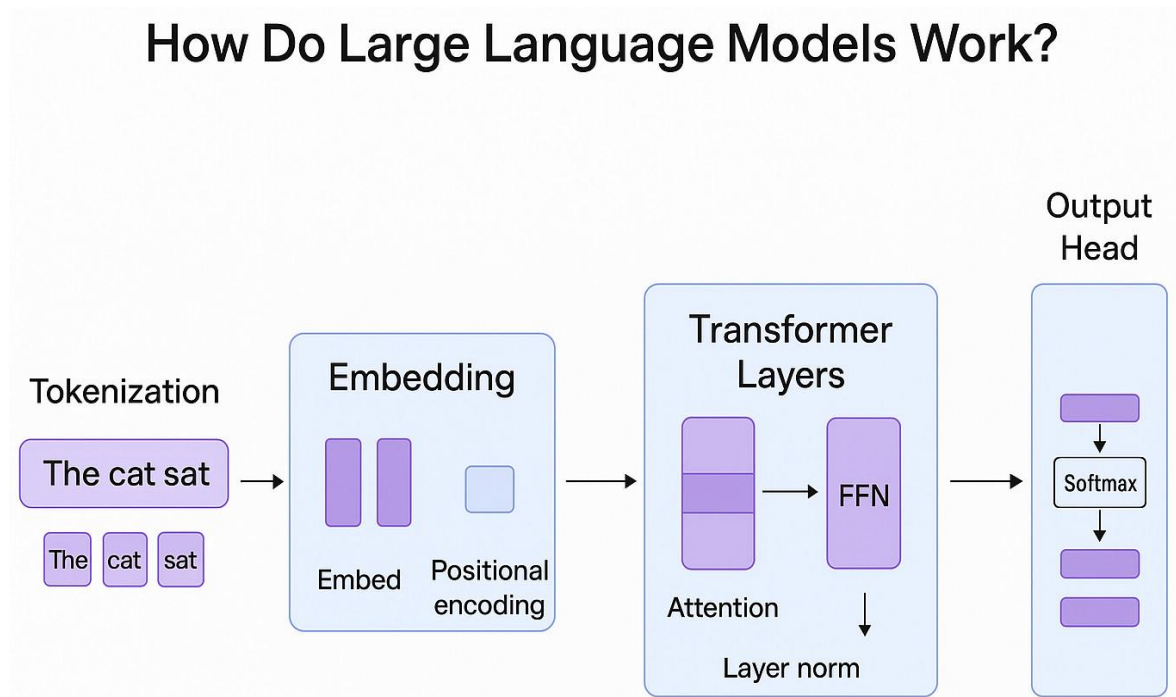
2.1. Định nghĩa và các khái niệm cốt lõi

2.1.1. Mô hình ngôn ngữ lớn (Large Language Models – LLM)

Mô hình ngôn ngữ lớn (Large Language Models – LLM) là các mô hình trí tuệ nhân tạo được huấn luyện trên khối lượng dữ liệu văn bản khổng lồ nhằm học các quy luật ngôn ngữ tự nhiên và sinh ra nội dung có ý nghĩa tương tự con người. Các mô hình này được xây dựng dựa trên kiến trúc Transformer và có khả năng thực hiện nhiều tác vụ như hiểu ngôn ngữ, tóm tắt văn bản, trả lời câu hỏi, sinh mã nguồn, phân tích tài liệu và hỗ trợ ra quyết định.

Trong lĩnh vực kỹ nghệ phần mềm, LLM không chỉ hỗ trợ lập trình mà còn được ứng dụng trong nhiều hoạt động của quy trình đảm bảo chất lượng phần mềm như phân tích yêu cầu, xây dựng tài liệu kiểm thử, sinh dữ liệu kiểm thử và tạo ca kiểm thử tự động. Các mô hình tiêu biểu hiện nay bao gồm GPT (OpenAI), Gemini (Google), Claude (Anthropic) và Llama (Meta).

Sự phát triển của LLM đã tạo ra bước tiến quan trọng trong việc tự động hóa các công việc dựa trên ngôn ngữ tự nhiên, đặc biệt là các nhiệm vụ vốn phụ thuộc nhiều vào kinh nghiệm của chuyên gia kiểm thử.



Hình 2.1: Kiến trúc tổng quát của mô hình ngôn ngữ lớn (LLM)

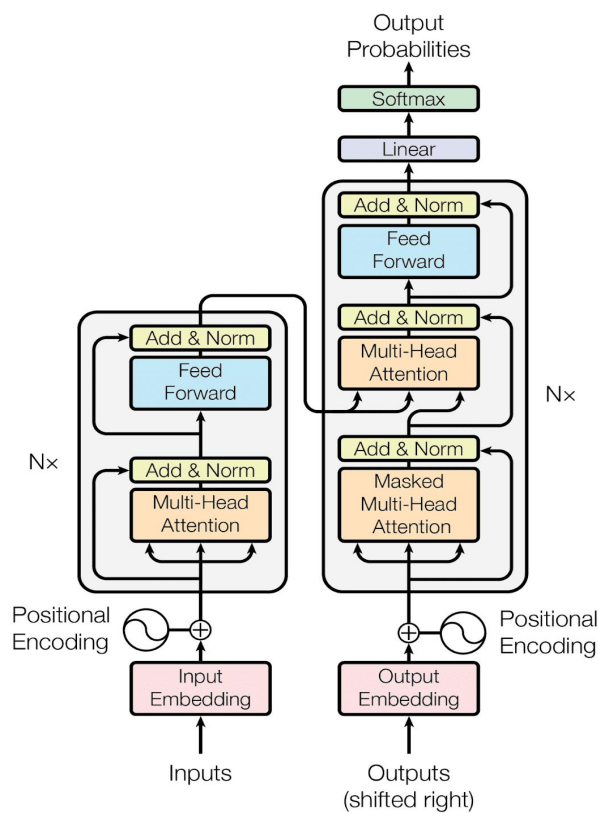
2.1.2. Kiến trúc Transformer và nguyên lý hoạt động của LLM

Transformer là kiến trúc nền tảng của hầu hết các mô hình ngôn ngữ lớn hiện nay. Khác với các mô hình tuần tự truyền thống như RNN hoặc LSTM, Transformer cho phép xử lý dữ liệu song song, giúp tăng tốc độ huấn luyện và cải thiện khả năng học ngữ cảnh dài hạn.

Thành phần quan trọng nhất của Transformer là cơ chế Self-Attention. Cơ chế này cho phép mô hình xác định mức độ liên quan giữa các từ trong câu để hiểu chính xác ý nghĩa của văn bản.

Trong quá trình xử lý, văn bản đầu vào được chia thành các Token. Các Token được chuyển đổi thành dạng số học và đưa vào mô hình để thực hiện suy luận. Kết quả đầu ra được tạo ra dựa trên xác suất xuất hiện của các Token tiếp theo trong chuỗi ngữ cảnh.

Nhờ khả năng học ngữ nghĩa ở quy mô lớn, LLM có thể hiểu các tài liệu yêu cầu phần mềm, nhận diện quy tắc nghiệp vụ và hỗ trợ sinh các ca kiểm thử tương ứng.



Hình 2.2: Kiến trúc Transformer

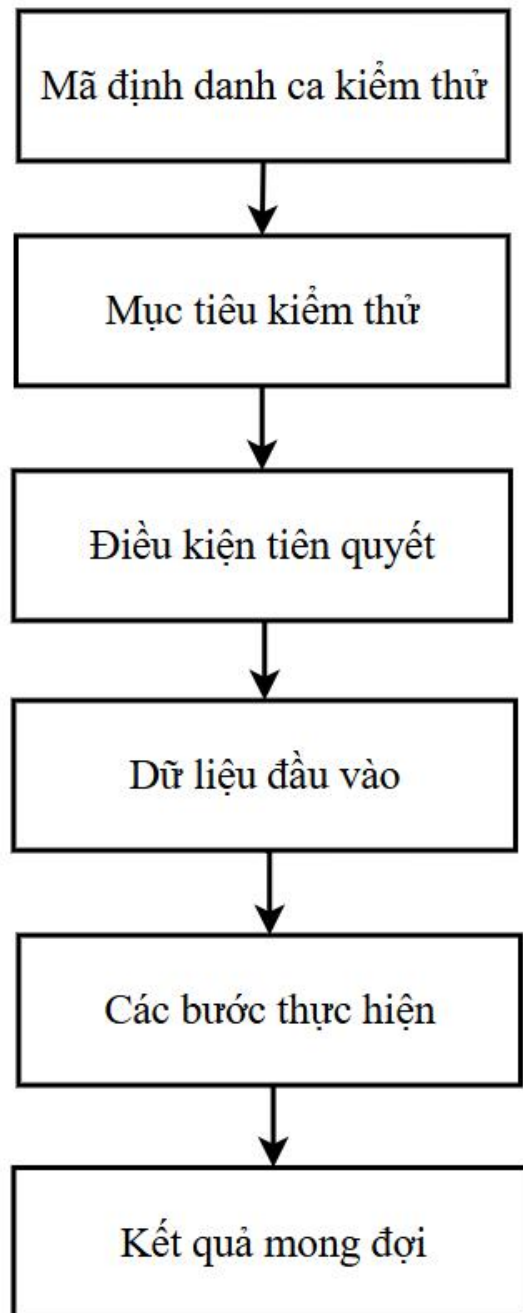
2.1.3. Phân tích yêu cầu phần mềm (Requirements Analysis)

- ❖ Phân tích yêu cầu là giai đoạn đầu tiên và có vai trò nền tảng trong vòng đời phát triển phần mềm. Mục tiêu của hoạt động này là thu thập, làm rõ, xác thực và tài liệu hóa các nhu cầu của khách hàng thành các yêu cầu cụ thể cho hệ thống.
- ❖ Yêu cầu phần mềm thường được chia thành hai nhóm:
 - Yêu cầu chức năng (Functional Requirements): mô tả các chức năng mà hệ thống phải thực hiện.
 - Yêu cầu phi chức năng (Non-functional Requirements): mô tả các thuộc tính chất lượng như hiệu năng, bảo mật, khả năng mở rộng và tính sẵn sàng.
- ❖ Trong các dự án Agile hiện đại, yêu cầu thường được mô tả dưới dạng User Story hoặc Use Case. Đây là nguồn thông tin quan trọng để xây dựng các ca kiểm thử và đánh giá tính đúng đắn của hệ thống.
- ❖ Tuy nhiên, tài liệu yêu cầu thường tồn tại các vấn đề như thiếu thông tin, mơ hồ hoặc không nhất quán. Đây là nguyên nhân dẫn đến nhiều lỗi nghiêm trọng trong quá trình phát triển phần mềm. Vì vậy, việc ứng dụng LLM để hỗ trợ phân tích yêu cầu đang trở thành hướng nghiên cứu và ứng dụng có nhiều tiềm năng.

2.1.4. Kiểm thử phần mềm và ca kiểm thử

- ❖ Kiểm thử phần mềm là một cuộc điều tra kỹ thuật được thực hiện nhằm cung cấp thông tin liên quan đến chất lượng của sản phẩm phần mềm cho các bên liên quan. Nó không chỉ là việc kiểm tra xem phần mềm có hoạt động hay không, mà còn là quá trình tìm hiểu xem liệu thứ chúng ta xây dựng có thực sự thực hiện đúng những gì được yêu cầu hay không.
- ❖ Ca kiểm thử (Test Case) là đơn vị cơ bản của hoạt động kiểm thử, được sử dụng để xác minh một yêu cầu hoặc chức năng cụ thể của hệ thống. Mỗi ca kiểm thử bao gồm các thành phần cơ bản như:
 - Mã định danh ca kiểm thử.
 - Mục tiêu kiểm thử.
 - Điều kiện tiên quyết.
 - Dữ liệu đầu vào.

- Các bước thực hiện.
 - Kết quả mong đợi.
- ❖ Chất lượng của ca kiểm thử có ảnh hưởng trực tiếp đến hiệu quả phát hiện lỗi và độ bao phủ kiểm thử. Do đó, việc tự động sinh ca kiểm thử bằng LLM được xem là giải pháp giúp giảm đáng kể thời gian xây dựng test case và nâng cao hiệu quả kiểm thử.



Hình 2.3: Cấu trúc cơ bản của một ca kiểm thử (Test Case)

2.1.5. Các loại hiện vật kiểm thử (Test Artifacts)

Trong quá trình kiểm thử, nhiều loại tài liệu được tạo ra nhằm hỗ trợ quản lý và theo dõi chất lượng phần mềm, bao gồm:

- ❖ **Đặc tả yêu cầu (Requirements Specification):** Là tài liệu văn bản mô tả các tính năng và hành vi dự kiến của ứng dụng. Đây là thành phần quan trọng nhất để LLM phân tích ngữ nghĩa nhằm phát hiện các sai sót hoặc mơ hồ ngay từ giai đoạn đầu.
- ❖ **Đặc tả kiểm thử (Test Specification):** Bản tóm tắt chi tiết về kịch bản, cách thức và tần suất kiểm thử cho một tính năng cụ thể. LLM có thể dựa trên cấu trúc này để tự động sinh ra các kịch bản kiểm thử tương ứng.
- ❖ **Kịch bản kiểm thử (Test Script):** Các hướng dẫn từng dòng để thực thi kiểm thử tự động, thường được viết bằng các ngôn ngữ lập trình như Java hay C#.
- ❖ **Báo cáo kiểm thử (Software Test Report):** Chứa tóm tắt các hoạt động và kết quả cuối cùng sau khi thực thi.
- ❖ **Ma trận truy xuất nguồn gốc (Traceability Matrix):** Một tài liệu bảng biểu giúp tương quan và truy vết danh sách yêu cầu ban đầu với việc triển khai kiểm thử và kết quả thực tế.

Các hiện vật này đồng thời cũng là nguồn dữ liệu đầu vào quan trọng để LLM phân tích, suy luận và sinh ra các ca kiểm thử phù hợp với yêu cầu nghiệp vụ.

2.2. Phân tích chuyên sâu

2.2.1. Cơ chế hoạt động của LLM trong phân tích yêu cầu

- ❖ Khả năng xử lý ngôn ngữ tự nhiên là yếu tố giúp LLM trở thành công cụ hữu ích trong phân tích yêu cầu phần mềm.
- ❖ Khi nhận đầu vào là tài liệu yêu cầu hoặc User Story, mô hình có thể:
 - Trích xuất các chức năng chính của hệ thống.
 - Xác định tác nhân và mục tiêu nghiệp vụ.
 - Nhận diện các điều kiện đầu vào và đầu ra.
 - Phát hiện các yêu cầu mơ hồ hoặc chưa đầy đủ.
 - Tóm tắt và chuẩn hóa tài liệu yêu cầu.

- ❖ Thông qua quá trình này, LLM giúp giảm đáng kể khối lượng công việc thủ công của Business Analyst và Tester, đồng thời nâng cao khả năng phát hiện sai sót ngay từ giai đoạn đầu của dự án.

2.2.2. Cơ chế sinh ca kiểm thử bằng LLM

- ❖ Sau khi phân tích yêu cầu, LLM có thể tự động sinh ra các ca kiểm thử dựa trên các quy tắc và ngữ cảnh đã được nhận diện.
- ❖ Quá trình sinh ca kiểm thử bằng LLM thường được thực hiện theo các bước:
 - Bước 1: Tiếp nhận tài liệu yêu cầu hoặc User Story.
 - Bước 2: Phân tích ngữ nghĩa và xác định quy tắc nghiệp vụ.
 - Bước 3: Trích xuất các điều kiện kiểm thử.
 - Bước 4: Xây dựng các trường hợp kiểm thử.
 - Bước 5: Sinh kết quả dưới dạng bảng Test Case.
- ❖ Các ca kiểm thử được sinh ra cần đảm bảo khả năng truy vết với yêu cầu gốc, đồng thời đáp ứng các tiêu chí về tính đầy đủ, tính đúng đắn và khả năng phát hiện lỗi. Trong thực tế, kết quả sinh ra thường được chuyên gia kiểm thử rà soát và hiệu chỉnh trước khi sử dụng chính thức.
- ❖ Các loại test case có thể được sinh tự động bao gồm:
 - Positive Test Cases.
 - Negative Test Cases.
 - Boundary Value Test Cases.
 - Exception Test Cases.
- ❖ Nhờ khả năng suy luận ngữ cảnh, LLM có thể đề xuất nhiều trường hợp kiểm thử mà người kiểm thử có thể bỏ sót khi thực hiện thủ công.

2.2.3. Prompt Engineering trong phân tích yêu cầu và sinh ca kiểm thử

Prompt Engineering là quá trình thiết kế và tối ưu hóa câu lệnh đầu vào nhằm hướng dẫn mô hình ngôn ngữ lớn (LLM) tạo ra kết quả chính xác, đầy đủ và phù hợp với mục tiêu sử dụng.

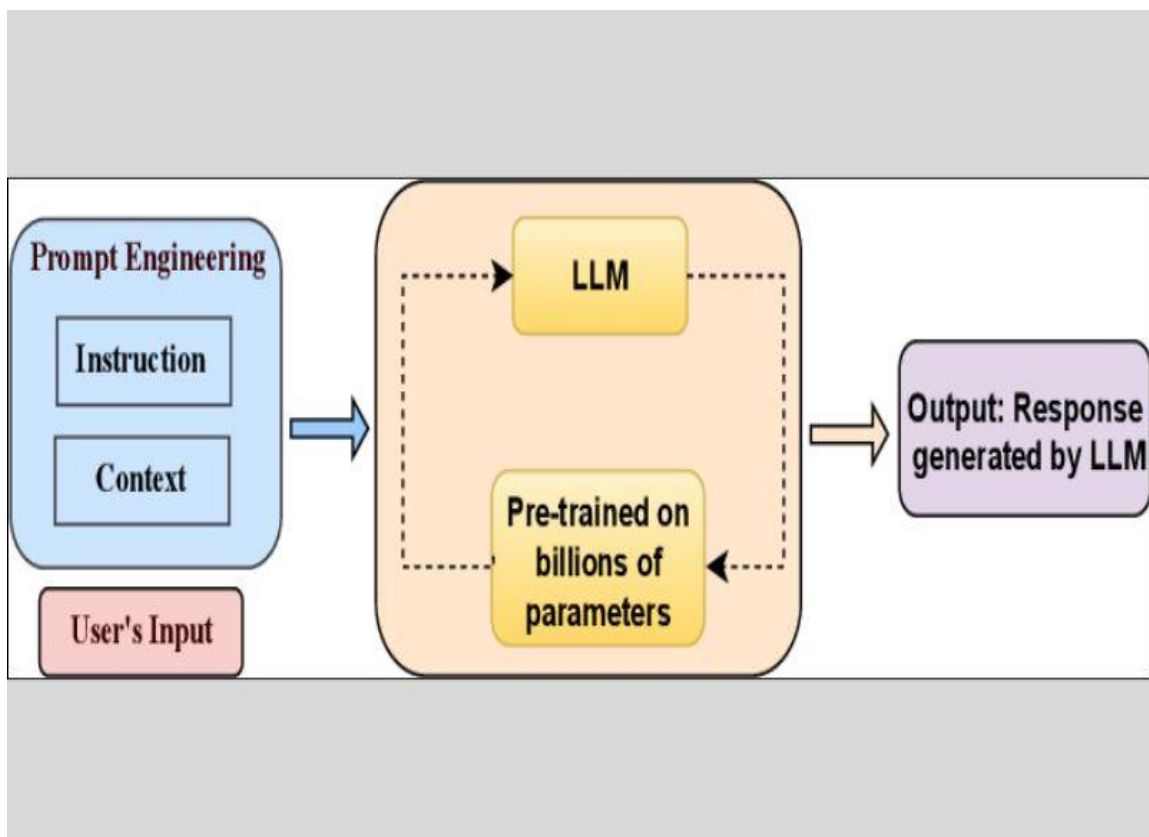
Trong phân tích yêu cầu phần mềm, Prompt Engineering hỗ trợ LLM trích xuất thông tin quan trọng, xác định chức năng hệ thống, nhận diện quy tắc nghiệp vụ và phát hiện các yêu cầu mơ hồ hoặc chưa hoàn chỉnh.

Trong sinh ca kiểm thử, Prompt Engineering giúp định hướng mô hình tạo ra các test case theo cấu trúc mong muốn, bao gồm điều kiện đầu vào, các bước thực hiện và kết quả mong đợi. Đồng thời, mô hình có thể sinh nhiều loại kiểm thử khác nhau như Positive Test Case, Negative Test Case, Boundary Value Test Case và Exception Test Case.

Các kỹ thuật Prompt Engineering phổ biến gồm:

- Zero-shot Prompting: Không cung cấp ví dụ mẫu.
- Few-shot Prompting: Cung cấp một số ví dụ để hướng dẫn mô hình.
- Chain-of-Thought Prompting: Yêu cầu mô hình suy luận theo từng bước.
- Role-based Prompting: Gán vai trò như Business Analyst, Tester hoặc QA Engineer cho mô hình.

Nhìn chung, Prompt Engineering đóng vai trò quan trọng trong việc nâng cao chất lượng phân tích yêu cầu và sinh ca kiểm thử, góp phần cải thiện hiệu quả ứng dụng LLM trong kiểm thử phần mềm.



Hình 2.4: Vai trò của Prompt Engineering trong ứng dụng LLM

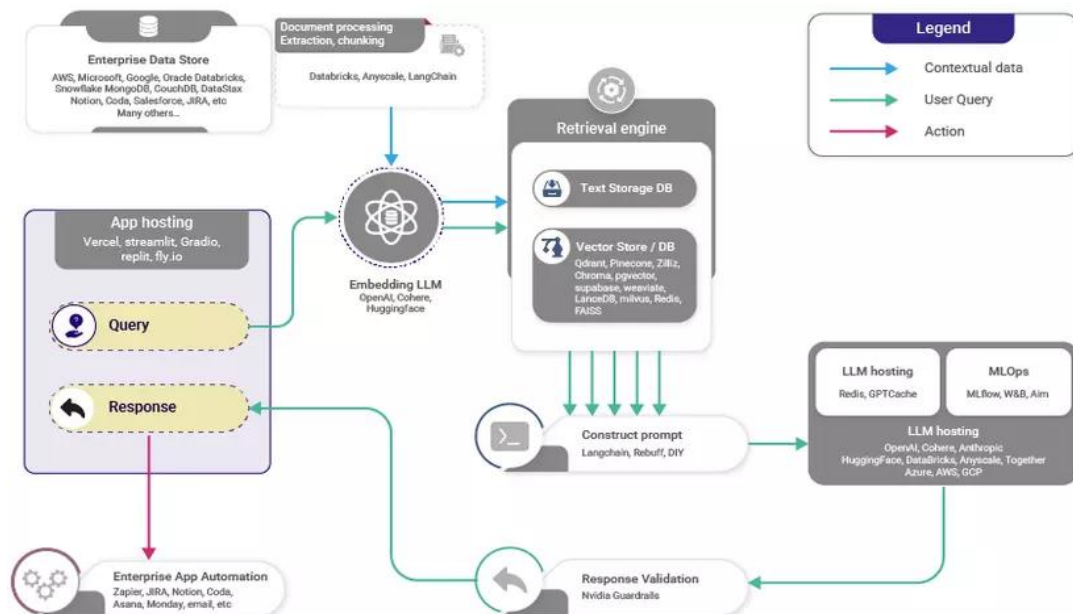
2.2.4. Retrieval-Augmented Generation (RAG) trong hỗ trợ kiểm thử phần mềm

Retrieval-Augmented Generation (RAG) là kỹ thuật kết hợp giữa khả năng truy xuất tri thức từ nguồn dữ liệu bên ngoài và khả năng sinh ngôn ngữ của LLM. Thay vì chỉ dựa trên kiến thức đã được huấn luyện, mô hình có thể truy xuất các tài liệu liên quan như đặc tả yêu cầu, User Story, Use Case hoặc tài liệu nghiệp vụ trước khi sinh kết quả.

Trong bài toán phân tích yêu cầu và sinh ca kiểm thử, RAG giúp:

- Tăng độ chính xác của thông tin được sinh ra.
- Giảm hiện tượng Hallucination.
- Đảm bảo test case phù hợp với tài liệu thực tế của dự án.
- Hỗ trợ cập nhật tri thức mới mà không cần huấn luyện lại mô hình.

Việc kết hợp RAG với LLM hiện được xem là một trong những hướng tiếp cận hiệu quả nhất trong các hệ thống hỗ trợ kiểm thử thông minh.



Hình 2.5: Kiến trúc Retrieval-Augmented Generation (RAG) trong hệ thống LLM

2.2.5. Ưu điểm của việc ứng dụng LLM

❖ Việc ứng dụng LLM trong phân tích yêu cầu và sinh ca kiểm thử mang lại nhiều lợi ích:

- Rút ngắn thời gian đọc và phân tích tài liệu.
- Tăng tốc độ xây dựng test case.

- Hỗ trợ phát hiện yêu cầu mơ hồ hoặc thiếu nhất quán.
 - Nâng cao độ bao phủ kiểm thử.
 - Hỗ trợ tester mới hoặc ít kinh nghiệm.
 - Giảm chi phí nhân lực cho hoạt động kiểm thử.
 - Tăng khả năng truy vết giữa yêu cầu và test case.
- ❖ Đặc biệt, trong các dự án có số lượng yêu cầu lớn, LLM giúp cải thiện đáng kể hiệu quả làm việc của nhóm QA.

2.2.6. Hạn chế và rủi ro của LLM

- ❖ Bên cạnh những lợi ích, việc ứng dụng LLM cũng tồn tại nhiều hạn chế:
- Hallucination (sinh thông tin không chính xác).
 - Hiểu sai ngữ cảnh nghiệp vụ.
 - Thiếu tính nhất quán giữa các lần sinh kết quả.
 - Phụ thuộc vào chất lượng Prompt.
 - Nguy cơ rò rỉ dữ liệu khi sử dụng dịch vụ AI công cộng.
- ❖ Do đó, kết quả sinh ra bởi LLM cần được kiểm chứng bởi chuyên gia nghiệp vụ hoặc tester trước khi đưa vào sử dụng chính thức.

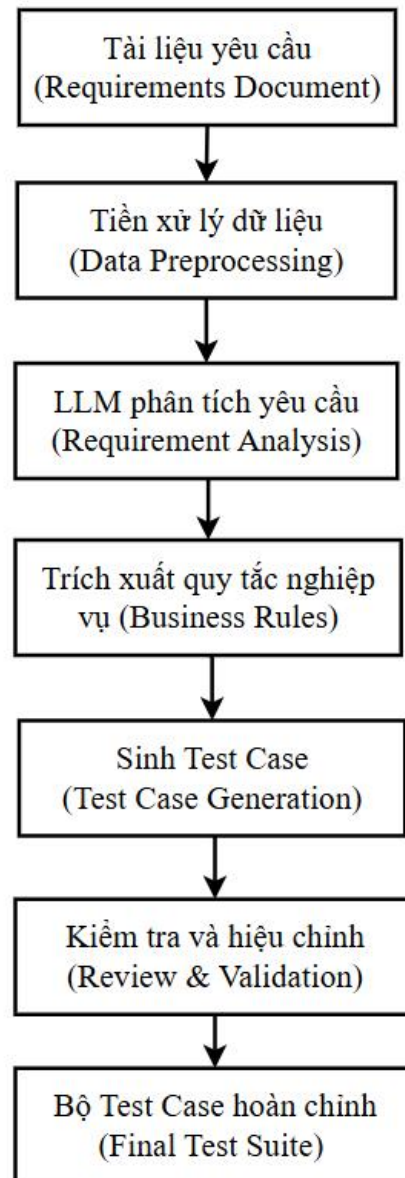
2.2.7. So sánh phương pháp truyền thống và phương pháp sử dụng LLM

Tiêu chí	Phương pháp truyền thống	Phương pháp sử dụng LLM
Phân tích yêu cầu	Thực hiện thủ công	Hỗ trợ tự động
Sinh Test Case	Tester xây dựng	Sinh tự động
Tốc độ xử lý	Chậm	Nhanh
Độ bao phủ	Phụ thuộc kinh nghiệm	Cao hơn
Chi phí nhân lực	Cao	Thấp hơn
Khả năng mở rộng	Hạn chế	Tốt
Rủi ro	Sai sót thủ công	Hallucination

Bảng 2.1. So sánh giữa phương pháp truyền thống và phương pháp sử dụng LLM

Bảng so sánh cho thấy LLM có khả năng cải thiện đáng kể năng suất làm việc của nhóm QA. Tuy nhiên, LLM chỉ đóng vai trò công cụ hỗ trợ và không thể thay thế hoàn toàn quy trình kiểm thử truyền thống; các kết quả sinh ra vẫn cần được xác thực bởi chuyên gia để đảm bảo tính chính xác và phù hợp với nghiệp vụ thực tế.

2.2.8. Quy trình ứng dụng LLM trong phân tích yêu cầu và sinh ca kiểm thử



Hình 2.6. Quy trình ứng dụng LLM trong phân tích yêu cầu và sinh ca kiểm thử

- ❖ Trong quy trình này:
 - Tài liệu yêu cầu đóng vai trò dữ liệu đầu vào.
 - LLM thực hiện phân tích ngữ nghĩa và suy luận nghiệp vụ.
 - Các điều kiện kiểm thử được trích xuất tự động.
 - Hệ thống sinh các ca kiểm thử phù hợp.
 - Tester thực hiện đánh giá và hiệu chỉnh kết quả trước khi sử dụng.
- ❖ Quy trình trên thể hiện rõ cách thức LLM được tích hợp vào hoạt động kiểm thử phần mềm hiện đại.

2.3. Sự liên hệ với chất lượng phần mềm

- ❖ **Nâng cao chất lượng yêu cầu:** LLM giúp phát hiện các yêu cầu mơ hồ, thiếu thông tin hoặc không nhất quán, từ đó cải thiện tài liệu đặc tả và giảm lỗi phát sinh.
- ❖ **Hỗ trợ Shift-Left Testing:** LLM cho phép thực hiện phân tích và phát hiện lỗi ngay từ giai đoạn đầu, giúp giảm chi phí sửa lỗi và nâng cao hiệu quả kiểm thử.
- ❖ **Tăng độ bao phủ kiểm thử:** LLM có khả năng sinh nhiều Test Case bao gồm trường hợp hợp lệ, không hợp lệ và ngoại lệ, giúp hạn chế bỏ sót lỗi.
- ❖ **Hỗ trợ kiểm thử liên tục:** LLM có thể hỗ trợ cập nhật Test Case, đánh giá tác động thay đổi và kết hợp với quy trình CI/CD để cải thiện hoạt động QA.
- ❖ **Nâng cao chất lượng phần mềm tổng thể:** LLM đóng vai trò là công cụ hỗ trợ tester, giúp tối ưu nguồn lực, giảm thời gian kiểm thử và nâng cao độ tin cậy của sản phẩm phần mềm.

Tóm tắt chương 2: Chương 2 đã trình bày các cơ sở lý thuyết về mô hình ngôn ngữ lớn (LLM), kiến trúc Transformer, phân tích yêu cầu phần mềm và kiểm thử phần mềm. Đồng thời, chương nghiên cứu cách LLM hỗ trợ phân tích yêu cầu, sinh Test Case, vai trò của Prompt Engineering và RAG trong việc nâng cao độ chính xác. Bên cạnh đó, chương cũng phân tích ưu điểm, hạn chế của LLM và sự đóng góp của công nghệ này trong việc nâng cao chất lượng, hiệu quả của quy trình kiểm thử phần mềm.

CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ

3.1. Mô tả bài toán thực tế

3.1.1. Bài toán thực tế

Trong các hệ thống phần mềm hiện đại, chức năng đăng nhập (Login) được xem là công kiểm soát truy cập đầu tiên, đóng vai trò xác thực danh tính người dùng trước khi cho phép truy cập vào các tài nguyên và dịch vụ bên trong hệ thống. Hầu hết các ứng dụng phổ biến như cổng thông tin sinh viên, hệ thống ngân hàng điện tử, nền tảng thương mại điện tử, hệ thống quản trị doanh nghiệp hay các dịch vụ trực tuyến đều yêu cầu cơ chế xác thực này nhằm đảm bảo tính an toàn và phân quyền truy cập hợp lý.

Trong trường hợp chức năng đăng nhập tồn tại lỗi, hệ thống có thể phải đối mặt với nhiều rủi ro nghiêm trọng. Cụ thể, kẻ tấn công có thể lợi dụng các lỗ hổng để truy cập trái phép vào dữ liệu nội bộ, dẫn đến nguy cơ rò rỉ thông tin người dùng và các tài nguyên quan trọng của hệ thống. Bên cạnh đó, sự cố liên quan đến xác thực còn làm suy giảm mức độ an toàn thông tin, gia tăng nguy cơ bị tấn công mạng, đồng thời ảnh hưởng trực tiếp đến uy tín và hình ảnh của tổ chức hoặc doanh nghiệp vận hành hệ thống.

Do đó, chức năng đăng nhập luôn được xem là một trong những thành phần trọng yếu và được ưu tiên kiểm thử kỹ lưỡng trong quá trình phát triển phần mềm thực tế. Việc đảm bảo tính đúng đắn và an toàn của cơ chế xác thực góp phần quan trọng trong việc nâng cao độ tin cậy của toàn bộ hệ thống.

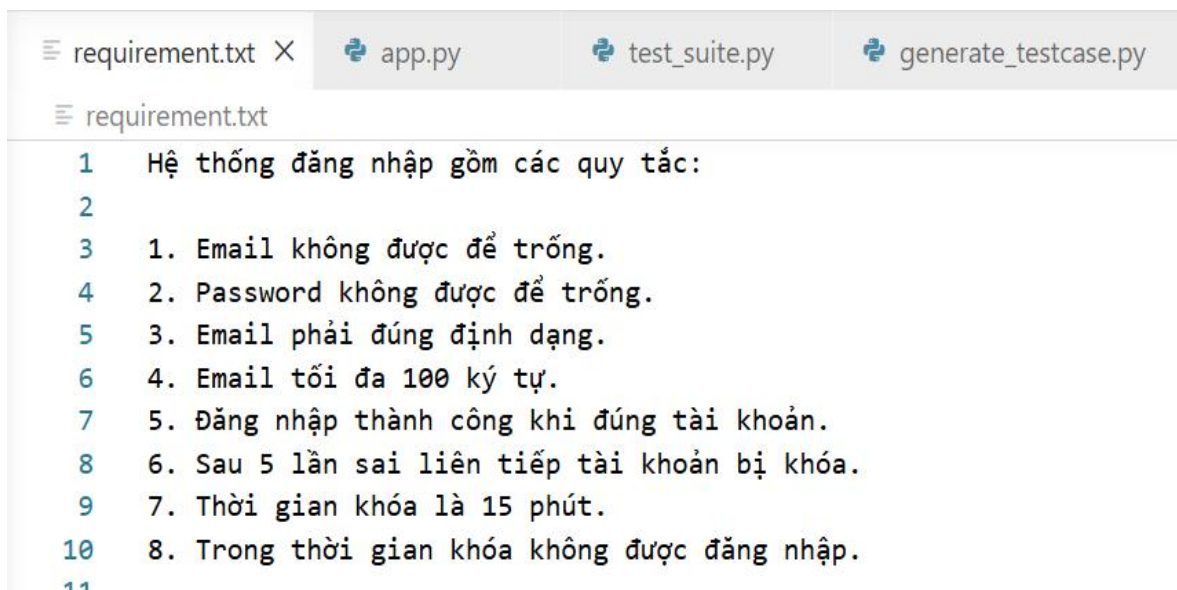
Trong phạm vi đề tài này, em lựa chọn xây dựng một hệ thống đăng nhập đơn giản sử dụng Flask API, nhằm mục đích minh họa quá trình ứng dụng mô hình ngôn ngữ lớn (LLM) trong việc phân tích yêu cầu hệ thống và tự động sinh các ca kiểm thử, từ đó hỗ trợ nâng cao hiệu quả kiểm thử phần mềm.

3.1.2. Đặc tả yêu cầu hệ thống

❖ Yêu cầu nghiệp vụ được mô tả dưới dạng ngôn ngữ tự nhiên trong file requirement.txt như sau:

- Email không được để trống.

- Password không được để trống.
 - Email phải đúng định dạng.
 - Email tối đa 100 ký tự.
 - Đăng nhập thành công khi đúng tài khoản.
 - Sau 5 lần đăng nhập sai liên tiếp tài khoản bị khóa.
 - Thời gian khóa là 15 phút.
 - Trong thời gian khóa không được phép đăng nhập.
- ❖ Đây là dạng tài liệu đặc tả thường gặp trong thực tế, được cung cấp bởi khách hàng hoặc bộ phận phân tích nghiệp vụ.

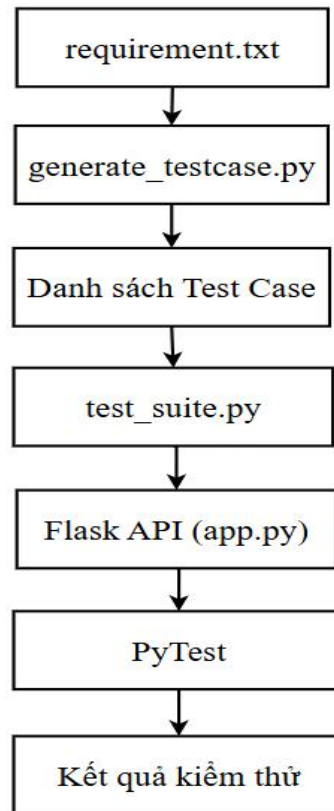


```
requirement.txt X app.py test_suite.py generate_testcase.py
requirement.txt
1  Hệ thống đăng nhập gồm các quy tắc:
2
3  1. Email không được để trống.
4  2. Password không được để trống.
5  3. Email phải đúng định dạng.
6  4. Email tối đa 100 ký tự.
7  5. Đăng nhập thành công khi đúng tài khoản.
8  6. Sau 5 lần sai liên tiếp tài khoản bị khóa.
9  7. Thời gian khóa là 15 phút.
10 8. Trong thời gian khóa không được đăng nhập.
11
```

Hình 3.1: Tài liệu yêu cầu đầu vào

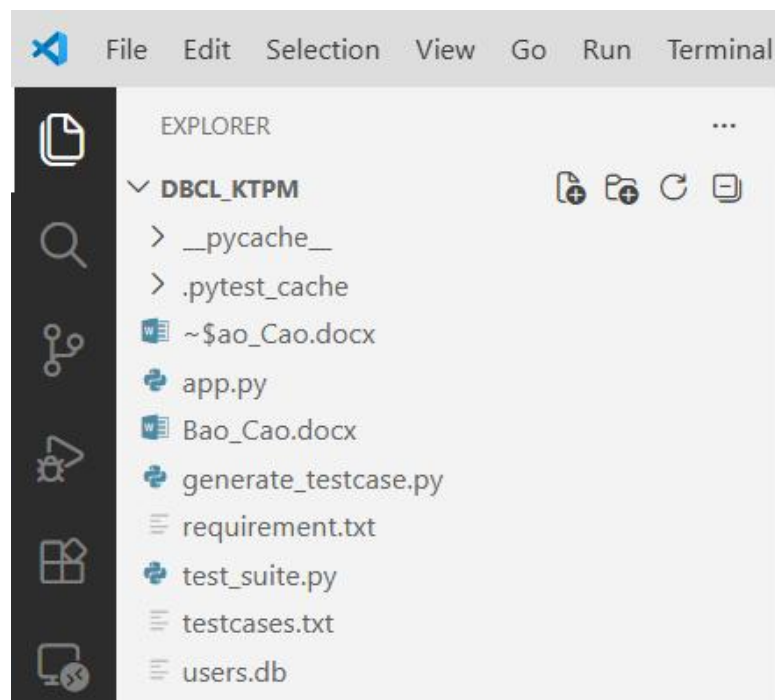
3.1.3. Kiến trúc hệ thống thực nghiệm

Để minh họa quá trình ứng dụng LLM trong phân tích yêu cầu và sinh ca kiểm thử, em đã xây dựng một hệ thống thực nghiệm gồm bốn thành phần chính: tài liệu yêu cầu (requirement.txt), module sinh ca kiểm thử (generate_testcase.py), hệ thống API đăng nhập (app.py) và bộ kiểm thử tự động (test_suite.py). Các thành phần này được liên kết thành một quy trình khép kín từ phân tích yêu cầu đến thực thi kiểm thử.



Hình 3.2. Kiến trúc hệ thống thực nghiệm

3.1.4. Cấu trúc thư mục dự án



Hình 3.3. Cấu trúc thư mục dự án

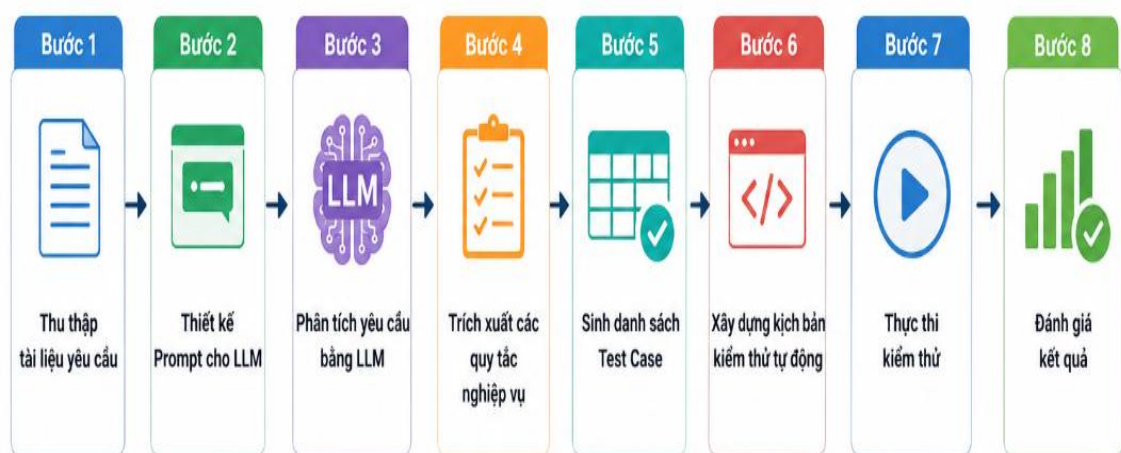
3.2. Áp dụng giải pháp

3.2.1. Quy trình áp dụng LLM

Quá trình áp dụng LLM trong đề tài được thực hiện theo các bước sau:

- ❖ Bước 1: Thu thập tài liệu yêu cầu.
- ❖ Bước 2: Thiết kế Prompt cho LLM.
- ❖ Bước 3: Phân tích yêu cầu bằng LLM.
- ❖ Bước 4: Trích xuất các quy tắc nghiệp vụ.
- ❖ Bước 5: Sinh danh sách Test Case.
- ❖ Bước 6: Xây dựng kịch bản kiểm thử tự động.
- ❖ Bước 7: Thực thi kiểm thử.
- ❖ Bước 8: Đánh giá kết quả.

Quy trình trên cho phép chuyển đổi từ tài liệu yêu cầu bằng ngôn ngữ tự nhiên thành bộ kiểm thử có thể thực thi được.



Hình 3.4: Quy trình áp dụng LLM

3.2.2. Thiết kế Prompt cho LLM

Để khai thác khả năng phân tích yêu cầu của LLM, đề tài sử dụng Prompt theo hướng Role-Based Prompting.

Prompt được thiết kế như sau:

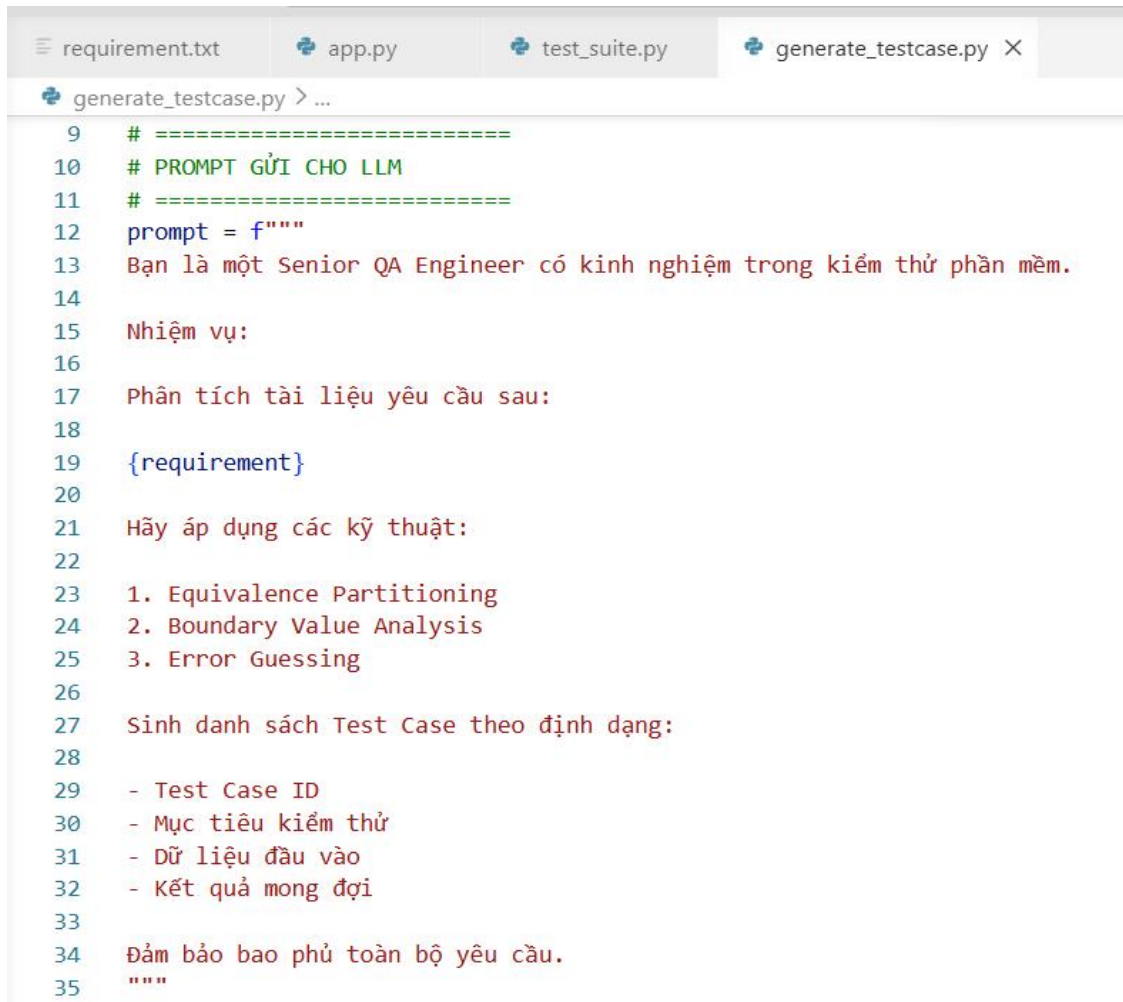
“Bạn là một QA Engineer có kinh nghiệm.

Hãy đọc tài liệu yêu cầu dưới đây.

- Xác định các quy tắc nghiệp vụ.
- Xác định các trường hợp kiểm thử.
- Xác định dữ liệu đầu vào.

- Xác định kết quả mong đợi.
- Sinh danh sách Test Case dưới dạng bảng.”

Prompt trên giúp định hướng mô hình đóng vai trò như một chuyên gia kiểm thử phần mềm.



```
9 # =====
10 # PROMPT GỬI CHO LLM
11 # =====
12 prompt = f"""
13 Bạn là một Senior QA Engineer có kinh nghiệm trong kiểm thử phần mềm.
14
15 Nhiệm vụ:
16
17 Phân tích tài liệu yêu cầu sau:
18
19 {requirement}
20
21 Hãy áp dụng các kỹ thuật:
22
23 1. Equivalence Partitioning
24 2. Boundary Value Analysis
25 3. Error Guessing
26
27 Sinh danh sách Test Case theo định dạng:
28
29 - Test Case ID
30 - Mục tiêu kiểm thử
31 - Dữ liệu đầu vào
32 - Kết quả mong đợi
33
34 Đảm bảo bao phủ toàn bộ yêu cầu.
35 """
```

Hình 3.5. Minh họa Prompt Engineering

3.2.3. Phân tích yêu cầu bằng LLM

- ❖ Sau khi nhận tài liệu yêu cầu, LLM tiến hành phân tích và xác định được các quy tắc nghiệp vụ sau:
 - R1: Email không được để trống.
 - R2: Password không được để trống.
 - R3: Email phải đúng định dạng.
 - R4: Email tối đa 100 ký tự.
 - R5: Đăng nhập thành công khi thông tin hợp lệ.
 - R6: Tài khoản bị khóa sau 5 lần đăng nhập sai.

- R7: Thời gian khóa là 15 phút.
 - R8: Người dùng không được đăng nhập khi tài khoản đang bị khóa.
- ❖ Kết quả cho thấy toàn bộ các yêu cầu đã được LLM nhận diện đầy đủ.

3.2.4. Xây dựng ma trận truy vết yêu cầu (Traceability Matrix)

- ❖ Sau khi xác định các yêu cầu, nhóm xây dựng ma trận truy vết giữa yêu cầu và Test Case.

Mã yêu cầu	Nội dung	Test Case
R1	Email không được để trống	TC002
R2	Password không được để trống	TC003
R3	Email đúng định dạng	TC004
R4	Email tối đa 100 ký tự	TC005
R5	Đăng nhập thành công	TC008
R6	Sai 5 lần liên tiếp	TC009
R7	Thời gian khóa 15 phút	TC010
R8	Không đăng nhập khi bị khóa	TC011

Bảng 3.1: Xây dựng ma trận truy vết yêu cầu

- ❖ Ma trận truy vết giúp đảm bảo mọi yêu cầu đều được kiểm thử.

3.2.5. Thiết kế bảng quyết định (Decision Table)

- ❖ Đối với chức năng đăng nhập, các điều kiện đầu vào gồm:
- Email tồn tại hay không.
 - Password đúng hay không.
- ❖ Bảng quyết định được xây dựng như sau:

Điều kiện	TC006	TC007	TC008
Email tồn tại	Không	Có	Có
Password đúng	Không	Không	Có
Kết quả	401	401	200

Bảng 3.2: Bảng quyết định

- ❖ Bảng quyết định giúp kiểm tra đầy đủ các khả năng xảy ra của hệ thống.

3.2.6. Thiết kế kiểm thử giá trị biên

- ❖ Đối với yêu cầu: Email tối đa 100 ký tự
- ❖ LLM xác định đây là bài toán kiểm thử giá trị biên.
- ❖ Các giá trị được lựa chọn:
 - 99 ký tự.
 - 100 ký tự.
 - 101 ký tự.
- ❖ Trong đó:
 - 99 và 100 ký tự là dữ liệu hợp lệ.
 - 101 ký tự là dữ liệu không hợp lệ.
- ❖ Việc kiểm thử giá trị biên giúp phát hiện các lỗi lập trình thường gặp liên quan đến điều kiện giới hạn.

3.2.7. Sinh Test Case tự động

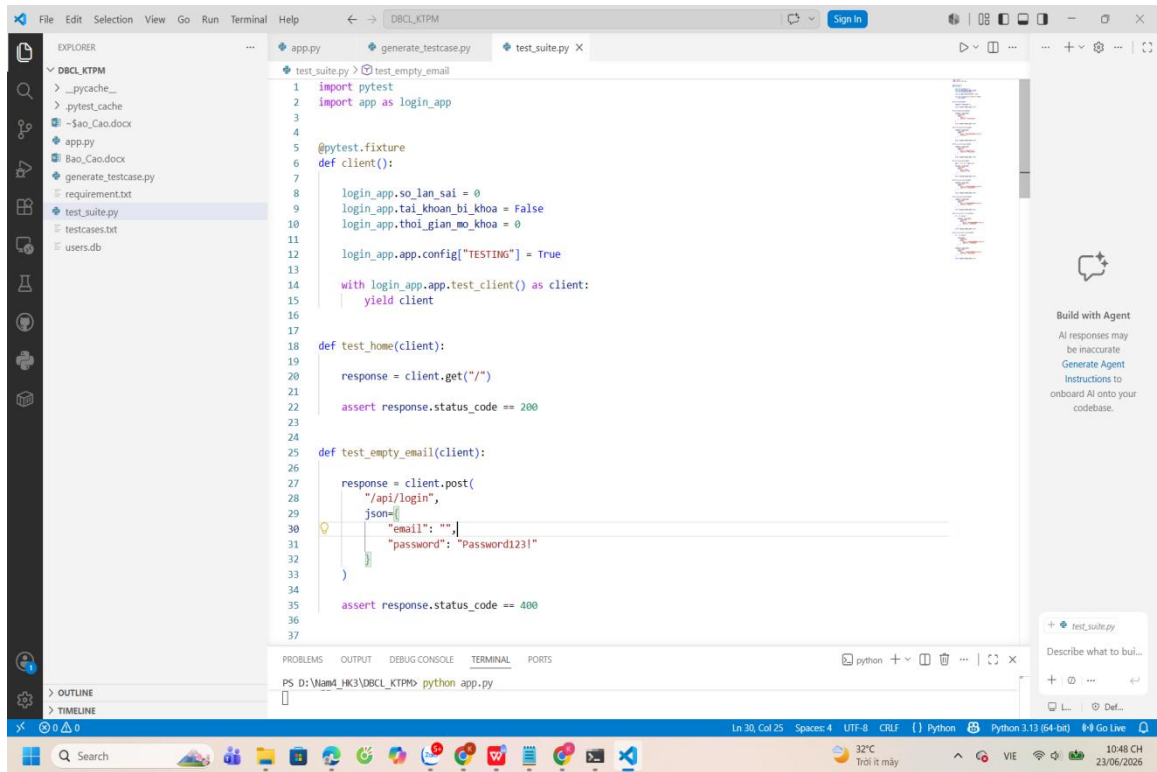
Dựa trên các quy tắc đã phân tích, LLM sinh ra danh sách Test Case:

Mã TC	Mục tiêu	Mã TC	Mục tiêu
TC001	Truy cập trang chủ	TC006	Đăng nhập thành công
TC002	Email rỗng	TC007	Sai mật khẩu
TC003	Password rỗng	TC008	Khóa tài khoản sau 5 lần sai
TC004	Email sai định dạng	TC009	Kiểm tra thời gian khóa
TC005	Email vượt quá 100 ký tự		

Bảng 3.3: Sinh Test Case tự động

3.2.8. Xây dựng kịch bản kiểm thử tự động

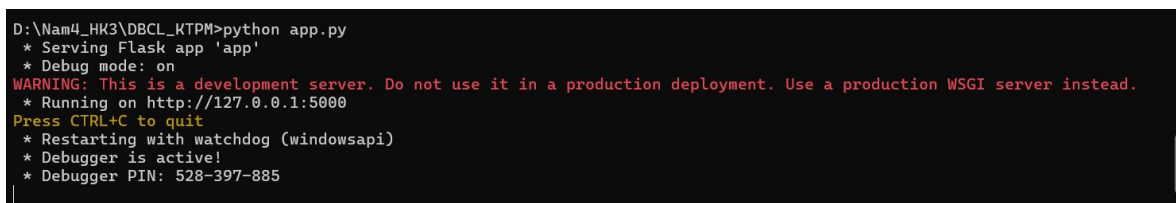
- ❖ Sau khi có danh sách Test Case, nhóm xây dựng file test_suite.py bằng thư viện PyTest.
- ❖ Mỗi Test Case được hiện thực thành một hàm kiểm thử riêng biệt.
- ❖ Ví dụ: test_empty_email(), test_invalid_email(), test_login_success().
- ❖ Việc sử dụng PyTest giúp tự động hóa quá trình kiểm thử và dễ dàng tích hợp vào quy trình CI/CD trong tương lai.



Hình 3.6. Cấu trúc bộ kiểm thử tự động

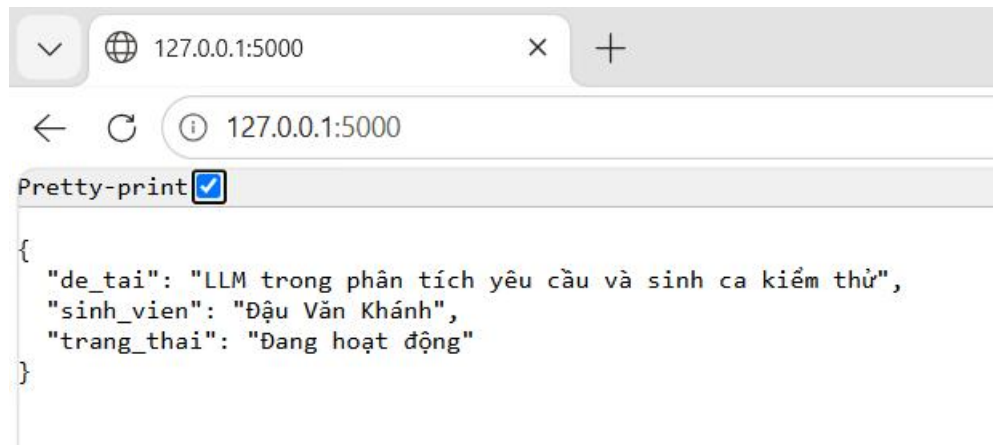
3.2.9. Thực thi kiểm thử và vận hành hệ thống

- ❖ Sau khi hoàn thành việc xây dựng hệ thống đăng nhập bằng Flask API và bộ kiểm thử tự động bằng PyTest, tiến hành triển khai hệ thống và thực hiện kiểm thử nhằm đánh giá mức độ đáp ứng các yêu cầu nghiệp vụ đã được đặc tả.
- ❖ Đầu tiên, hệ thống được khởi động bằng lệnh: `python app.py`
- ❖ Sau khi khởi động thành công, máy chủ Flask hoạt động tại địa chỉ: <http://127.0.0.1:5000>



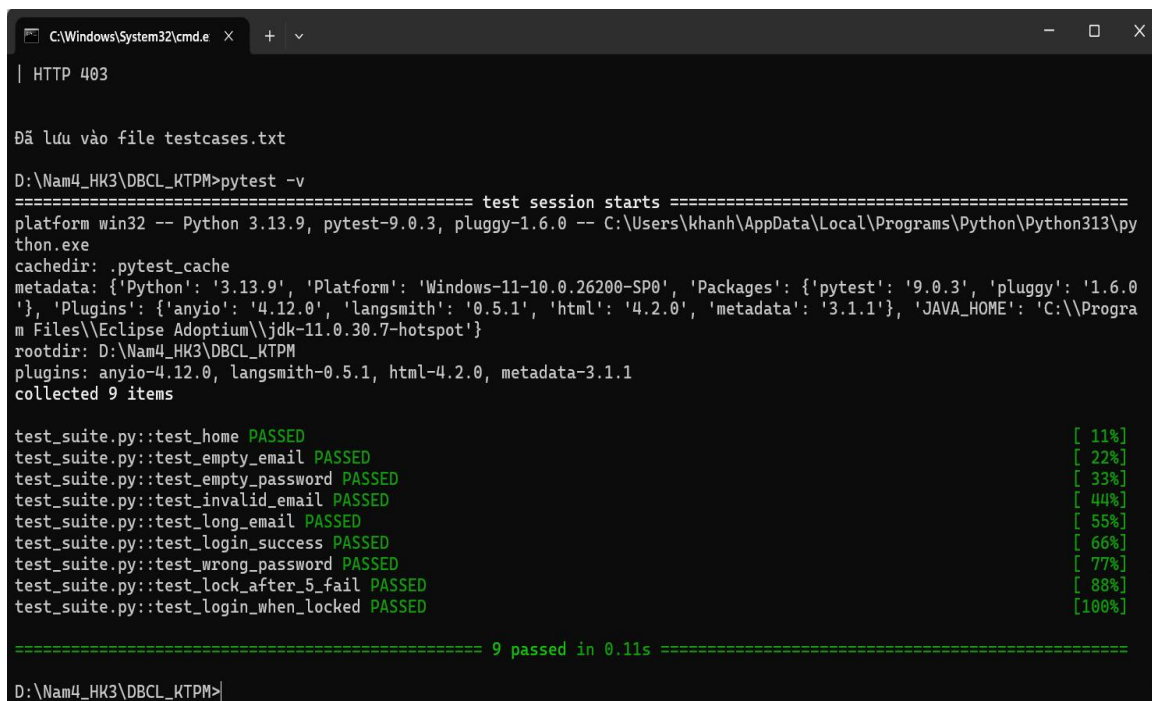
Hình 3.7. Kết quả khởi động hệ thống Flask API

- ❖ Tiếp theo, truy cập địa chỉ trên để kiểm tra trạng thái hoạt động của API. Hệ thống trả về dữ liệu JSON chứa thông tin của đề tài và trạng thái hoạt động, chứng tỏ API đã được triển khai thành công và sẵn sàng tiếp nhận các yêu cầu kiểm thử.



Hình 3.8. Kết quả truy cập API

- ❖ Sau khi xác nhận hệ thống hoạt động bình thường, bộ kiểm thử tự động được thực thi bằng lệnh: `pytest -v`
 - Bộ kiểm thử thực hiện xác minh các trường hợp như email rỗng, mật khẩu rỗng, email sai định dạng, email vượt quá giới hạn độ dài, email không tồn tại, sai mật khẩu và đăng nhập thành công.
 - Kết quả thực thi: 9 passed
 - Điều này cho thấy tất cả các chức năng hiện tại của hệ thống đều hoạt động đúng theo yêu cầu đã đặc tả.



Hình 3.9. Kết quả thực thi PyTest

3.3. Kết quả và đánh giá

3.3.1. Kết quả đạt được

Sau quá trình áp dụng mô hình ngôn ngữ lớn (LLM) vào bài toán phân tích yêu cầu và sinh ca kiểm thử, hệ thống đã đạt được các kết quả tích cực cả về mặt kỹ thuật lẫn hiệu quả kiểm thử.

Từ tài liệu yêu cầu được mô tả dưới dạng ngôn ngữ tự nhiên trong file requirement.txt, LLM đã hỗ trợ nhận diện và trích xuất đầy đủ các quy tắc nghiệp vụ liên quan đến chức năng đăng nhập. Trên cơ sở đó, hệ thống tự động đề xuất các trường hợp kiểm thử phù hợp, bao gồm các trường hợp hợp lệ, không hợp lệ và các trường hợp kiểm thử giá trị biên.

Kết quả thực nghiệm cho thấy bộ kiểm thử được sinh ra có khả năng bao phủ toàn bộ các yêu cầu chức năng của hệ thống. Đồng thời, các kịch bản kiểm thử tự động được xây dựng bằng PyTest có thể thực thi thành công và cho kết quả ổn định trong môi trường thử nghiệm.

Nhìn chung, việc ứng dụng LLM đã góp phần hỗ trợ đáng kể cho quá trình phân tích yêu cầu, thiết kế kiểm thử và xây dựng bộ kiểm thử tự động, giúp giảm khối lượng công việc thủ công của kỹ sư QA.

3.3.2. Đánh giá độ bao phủ yêu cầu

- ❖ Để đánh giá mức độ đáp ứng của bộ kiểm thử đối với hệ thống, đề tài tiến hành đối chiếu giữa các yêu cầu nghiệp vụ và các ca kiểm thử đã được xây dựng thông qua ma trận truy vết yêu cầu (Traceability Matrix).
- ❖ Kết quả thống kê cho thấy:
 - Tổng số yêu cầu nghiệp vụ được xác định: 8 yêu cầu
 - Tổng số yêu cầu đã được ánh xạ với Test Case: 8 yêu cầu
 - Tổng số Test Case được xây dựng và thực thi: 9 Test Case
- ❖ Tỷ lệ bao phủ yêu cầu được tính theo công thức:

$$Coverage = \frac{\text{Số yêu cầu được kiểm thử}}{\text{Tổng số yêu cầu}} \times 100\%$$

$$Coverage = 8/8 \times 100\% = 100\%$$

Kết quả cho thấy toàn bộ yêu cầu đều được kiểm thử.

3.3.3. So sánh hiệu quả trước và sau khi sử dụng LLM

Để đánh giá hiệu quả thực tế của việc ứng dụng LLM, đề tài tiến hành so sánh quy trình kiểm thử truyền thống với quy trình có sự hỗ trợ của LLM.

Tiêu chí	Phương pháp thủ công	Có hỗ trợ LLM
Phân tích yêu cầu	Thực hiện thủ công bởi tester	Hỗ trợ tự động bởi LLM
Thiết kế Test Case	Tự xây dựng	Sinh tự động
Viết Test Script	Viết từng trường hợp	Sinh từ Test Case
Khả năng bỏ sót yêu cầu	Cao	Thấp
Độ bao phủ kiểm thử	Trung bình	Cao
Khả năng tái sử dụng	Thấp	Cao

Bảng 3.4: So sánh hiệu quả trước và sau khi sử dụng LLM

Từ bảng so sánh có thể nhận thấy việc sử dụng LLM giúp rút ngắn đáng kể thời gian thực hiện các công đoạn trong quy trình kiểm thử. Đặc biệt, khả năng phân tích nhanh tài liệu yêu cầu và đề xuất các trường hợp kiểm thử phù hợp giúp nâng cao năng suất làm việc của kỹ sư QA.

Ngoài ra, LLM còn hỗ trợ phát hiện các trường hợp kiểm thử giá trị biên và các điều kiện đặc biệt mà người kiểm thử có thể bỏ sót khi thực hiện thủ công.

3.3.4. Đánh giá rủi ro và hạn chế

- ❖ Mặc dù đem lại nhiều lợi ích, việc ứng dụng LLM trong kiểm thử phần mềm vẫn tồn tại một số hạn chế nhất định:
 - Sinh Test Case chưa chính xác.
 - Hiểu sai yêu cầu phức tạp.
 - Bỏ sót các trường hợp ngoại lệ.
 - Phụ thuộc chất lượng Prompt.
- ❖ Vì vậy, trong giai đoạn hiện nay, LLM nên được xem là công cụ hỗ trợ cho kỹ sư kiểm thử thay vì thay thế hoàn toàn vai trò của con người.

3.3.5. Đánh giá chi phí và khả năng ứng dụng

- ❖ Từ kết quả thực nghiệm có thể nhận thấy việc ứng dụng LLM mang lại nhiều lợi ích về mặt chi phí và hiệu quả triển khai.
- ❖ Cụ thể:
 - Giảm thời gian phân tích yêu cầu.
 - Giảm công sức thiết kế Test Case.
 - Hỗ trợ sinh mã kiểm thử tự động.
 - Nâng cao độ bao phủ kiểm thử.
 - Tăng khả năng tái sử dụng trong các dự án khác.
 - Đẩy nhanh tiến độ phát triển phần mềm.
- ❖ Đối với các dự án có số lượng yêu cầu lớn hoặc thường xuyên thay đổi, việc sử dụng LLM có thể giúp tiết kiệm đáng kể nguồn lực kiểm thử, đồng thời hỗ trợ đội ngũ QA tập trung nhiều hơn vào các hoạt động kiểm thử chuyên sâu như kiểm thử bảo mật, kiểm thử hiệu năng và kiểm thử tích hợp.
- ❖ Kết quả nghiên cứu cho thấy mô hình ngôn ngữ lớn hoàn toàn có tiềm năng trở thành một công cụ hỗ trợ hiệu quả trong quy trình đảm bảo chất lượng phần mềm hiện đại, đặc biệt trong bối cảnh xu hướng ứng dụng trí tuệ nhân tạo vào phát triển phần mềm ngày càng được quan tâm và mở rộng.

Tóm tắt chương 3: Chương 3 đã trình bày quá trình vận dụng LLM vào bài toán thực tế là xây dựng và kiểm thử chức năng đăng nhập bằng Flask API. Nội dung chương tập trung vào việc sử dụng LLM để phân tích yêu cầu, trích xuất quy tắc nghiệp vụ, sinh Test Case, xây dựng ma trận truy vết và hỗ trợ tạo kịch bản kiểm thử tự động bằng PyTest. Kết quả thực nghiệm cho thấy bộ kiểm thử đáp ứng đầy đủ các yêu cầu chức năng với độ bao phủ đạt 100%, đồng thời việc ứng dụng LLM giúp giảm thời gian phân tích, nâng cao hiệu quả thiết kế kiểm thử và hỗ trợ cải thiện chất lượng phần mềm. Bên cạnh đó, chương cũng đánh giá các hạn chế và khả năng ứng dụng thực tế của LLM trong quy trình kiểm thử phần mềm.

CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ

4.1. Tổng kết nội dung nghiên cứu

Trong phạm vi tiểu luận, đề tài đã tập trung nghiên cứu và đánh giá khả năng ứng dụng của mô hình ngôn ngữ lớn (Large Language Model – LLM) trong lĩnh vực kiểm thử phần mềm, đặc biệt là việc hỗ trợ phân tích yêu cầu hệ thống và tự động sinh ca kiểm thử (Test Case Generation). Nghiên cứu hướng đến việc khai thác khả năng xử lý ngôn ngữ tự nhiên của LLM nhằm nâng cao hiệu quả trong các hoạt động đảm bảo chất lượng phần mềm.

Về mặt lý thuyết, đề tài đã trình bày các kiến thức nền tảng về mô hình ngôn ngữ lớn, kỹ thuật xây dựng Prompt (Prompt Engineering), các phương pháp kiểm thử phần mềm phổ biến và vai trò của trí tuệ nhân tạo trong quá trình tự động hóa kiểm thử. Qua đó, đề tài làm rõ tiềm năng của LLM trong việc hỗ trợ kỹ sư kiểm thử phân tích yêu cầu, thiết kế Test Case và nâng cao hiệu quả của quy trình phát triển phần mềm.

Về mặt thực nghiệm, đề tài đã xây dựng thành công hệ thống đăng nhập sử dụng Flask API nhằm mô phỏng một chức năng phần mềm thực tế. Trên hệ thống này, LLM được áp dụng để hỗ trợ quá trình kiểm thử thông qua các hoạt động như phân tích tài liệu yêu cầu, trích xuất quy tắc nghiệp vụ, sinh danh sách Test Case, xây dựng ma trận truy vết giữa yêu cầu và Test Case, đồng thời hỗ trợ xây dựng kịch bản kiểm thử tự động.

Kết quả thực nghiệm cho thấy việc ứng dụng LLM giúp giảm thời gian phân tích yêu cầu, hỗ trợ quá trình thiết kế bộ kiểm thử và nâng cao khả năng bao phủ của các ca kiểm thử. Qua đó, LLM được đánh giá là một công cụ hỗ trợ tiềm năng, góp phần nâng cao năng suất làm việc của kỹ sư kiểm thử và cải thiện chất lượng của quy trình đảm bảo chất lượng phần mềm.

4.2. Hạn chế của giải pháp

Mặc dù đạt được những kết quả tích cực, giải pháp vẫn còn một số hạn chế như sau:

- Phụ thuộc vào Prompt đầu vào: Chất lượng Test Case sinh ra từ LLM phụ thuộc nhiều vào cách xây dựng Prompt. Prompt chưa đầy đủ hoặc thiếu ngữ cảnh có thể dẫn đến kết quả chưa chính xác.

- Chưa thay thế hoàn toàn kỹ sư kiểm thử: LLM vẫn có thể gặp khó khăn trong việc xử lý các yêu cầu nghiệp vụ phức tạp, các trường hợp ngoại lệ hoặc các quy tắc đặc thù của hệ thống.
- Phạm vi thử nghiệm còn hạn chế: Đề tài mới được triển khai trên hệ thống đăng nhập có quy mô nhỏ, chưa đánh giá đầy đủ khả năng áp dụng trên các hệ thống lớn và phức tạp.
- Chưa mở rộng nhiều loại kiểm thử: Nghiên cứu chủ yếu tập trung vào kiểm thử chức năng, chưa xem xét các loại kiểm thử khác như kiểm thử hiệu năng, bảo mật và khả năng chịu tải.

4.3. Định hướng mở rộng

Dựa trên kết quả đạt được, đề tài có thể tiếp tục được mở rộng theo các hướng sau:

- Tích hợp với các mô hình LLM hiện đại như GPT, Gemini để tự động hóa quá trình phân tích yêu cầu và sinh Test Case.
- Tự động sinh mã kiểm thử và hỗ trợ nhiều framework như Selenium, Playwright, Robot Framework.
- Kết hợp LLM với CI/CD để tự động cập nhật và thực thi bộ kiểm thử khi hệ thống thay đổi.
- Nghiên cứu sinh dữ liệu kiểm thử tự động dựa trên tài liệu yêu cầu và quy tắc nghiệp vụ.
- Mở rộng đánh giá trên các hệ thống thực tế lớn như thương mại điện tử, ngân hàng, quản lý doanh nghiệp.
- Áp dụng kỹ thuật RAG nhằm nâng cao khả năng phân tích yêu cầu và độ chính xác của Test Case sinh ra từ LLM.

Tóm tắt chương: Chương 4 đã trình bày tổng kết kết quả nghiên cứu về việc ứng dụng LLM trong phân tích yêu cầu và sinh ca kiểm thử phần mềm. Kết quả cho thấy LLM có khả năng hỗ trợ hiệu quả trong việc phân tích tài liệu, xây dựng Test Case và nâng cao chất lượng kiểm thử. Tuy nhiên, giải pháp vẫn còn một số hạn chế như phụ thuộc vào Prompt, chưa thay thế hoàn toàn tester và phạm vi thử nghiệm còn giới hạn. Trong tương lai, đề tài có thể mở rộng bằng cách tích hợp các mô hình LLM hiện đại, tự động hóa kiểm thử và áp dụng trên các hệ thống thực tế lớn hơn.

TÀI LIỆU THAM KHẢO

1. Sahar Tahvili, Leo Hatvani, Artificial Intelligence Methods for Optimization of the Software Testing Process, Academic Press, 2022. (Tham khảo Chương 2: trang 44-60; Chương 4: trang 77-123).
2. Matthew Heusser, Michael Larsen, Software Testing Strategies, Packt Publishing, 2023. (Tham khảo Chương 6: trang 97-176).
3. Kiểm thử LLM: Các kỹ thuật và phương pháp thực hành tốt nhất mới nhất <https://www.patronus.ai/llm-testing>
4. Sử dụng LLM trong kiểm thử tự động: Cách tích hợp và tối ưu hóa <https://www.linkedin.com/pulse/using-llms-automation-testing-how-integrate-optimize-ajay-kulkarni-dvjxf>
5. LLM là gì? Tìm hiểu chi tiết về mô hình ngôn ngữ lớn: <https://fpt.ai/vi/bai-viet/mo-hinh-ngon-ngu-lon/>
6. LLM là gì? Khám phá ứng dụng và xu hướng của mô hình ngôn ngữ lớn: <https://vnptai.io/vi/blog/detail/llm-la-gi>
7. M. Tasarsu, A. V. Tokmak, and C. Catal, "Test case generation using large language models: a systematic literature review," Cluster Computing, vol. 29, no. 227, Springer, 2026. [Online]. Available: <https://link.springer.com/article/10.1007/s10586-026-06021-z>
8. A Review of Large Language Models for Automated Test Case Generation: <https://www.mdpi.com/2504-4990/7/3/97>
9. S. Masuda et al., "Generating High-Level Test Cases from Requirements using LLM: An Industry Study," arXiv preprint arXiv:2510.03641, 2025. [Online]. Available: <https://arxiv.org/abs/2510.03641>